

SIPaaS

SIP as a Service Platform

Complete Technical Summary & Local Setup Guide

Multi-tenant Vobiz-style SIP Trunk + DID Management Platform

Built on Kamailio · Asterisk · RTPEngine · MySQL · Redis

Stack	Kamailio 5.7 + Asterisk 18 + RTPEngine + Node.js API + Next.js Portal
Database	MySQL 8.0 + Redis 7
Upstream	Vocallabs / TATA (SIP trunks)
Monitoring	Prometheus + Grafana
Version	Production-ready Docker Compose stack

Table of Contents

Section#	Topic	Page
1	What is SIPaaS?	3
2	Architecture Overview	3
3	Component Reference	4
4	Outbound Call Flow (Step-by-Step)	6
5	Inbound Call Flow (Step-by-Step)	7
6	Customer Journey	7
7	Upstream Trunk Types	8
8	Debugging Session — What We Fixed	8
9	Port Reference	10
10	Local Setup — Step-by-Step Guide	11
11	Softphone Configuration	14
12	Admin Commands	14
13	Firewall Rules	15
14	Troubleshooting	15

Section 1 — What is SIPaaS?

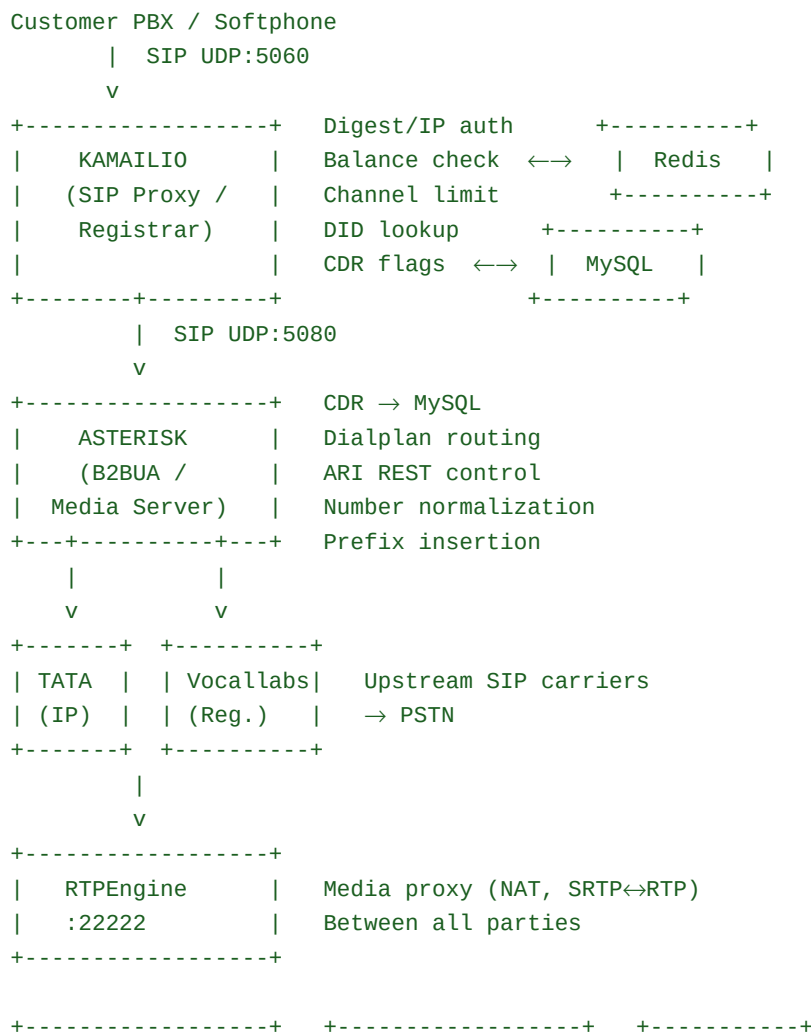
SIPaaS is a **multi-tenant SIP-as-a-Service platform** modelled on commercial services like Vobiz. It lets you act as a VoIP provider: customers sign up, buy SIP trunks and DID (Direct Inward Dialing) numbers, configure their own PBX or softphone, and make/receive PSTN calls — all billed per minute from their prepaid balance.

The platform handles everything: SIP registration, digest authentication, per-customer channel limits, real-time balance checking, CDR generation, DID routing, upstream carrier management, and a full customer-facing web portal.

Core value proposition:

Customer buys a SIP trunk → gets credentials → configures their PBX → calls route through your platform → billed at ■0.45/min (configurable) → you pay Vocallabs/TATA wholesale rates.

Section 2 — Architecture Overview



REST API		Customer Portal		Billing	
Node.js :3000		Next.js :8080		Engine	
JWT auth		Dashboard		60s loop	
+-----+		+-----+		+-----+	
+-----+		+-----+			
Prometheus		Grafana			
:9090		:3001			
+-----+		+-----+			

Section 3 — Component Reference

Component	Role	Port	Technology
Kamailio	SIP proxy, registrar, auth, billing gate, DID routing	5060 UDP/TCP	Kamailio 5.7
Asterisk	B2BUA, dialplan, CDR, ARI, upstream trunk management	5060 UDP/TCP	Asterisk 18
RTPEngine	Media proxy — NAT traversal, SRTP/RTP bridging	2222 UDP (ctrl)	RTPEngine
MySQL	All persistent data: subscribers, CDR, DID, billing	3307 (host)	MySQL 8.0
Redis	Real-time balance cache, channel counters	6379 (internal)	Redis 7
REST API	JWT-authenticated REST API for portal and integration	3000 TCP	Node.js / Express
Portal	Customer-facing web dashboard	8080 TCP	Next.js
Billing Engine	CDR rating (per-minute), DID monthly renewals	internal	Node.js
Prometheus	Metrics collection	9090 TCP	Prometheus
Grafana	Monitoring dashboards	3001 TCP	Grafana

3.1 Kamailio — SIP Edge

Kamailio is the first point of contact for all customer SIP traffic. It handles:

- SIP REGISTER — stores customer location in the *location* table (usrloc module)
- Digest authentication — validates username/password against the *subscriber* table
- IP authentication — allows trusted customer IPs without digest (address table, group 1)
- Balance check — reads **sipaas:balance:{customer_id}** from Redis; rejects with 402 if zero
- Channel limit — reads **sipaas:channels:{customer_id}** from Redis; rejects with 503 if at limit
- Dispatcher — load-balances calls to Asterisk nodes (setid=1 in dispatcher table)
- DID routing — for inbound calls, maps called number to customer via *did_numbers* table
- CDR flags — sets accounting flags so acc module records call start/end to MySQL
- RTPEngine offer — rewrites SDP before forwarding to Asterisk

3.2 Asterisk — B2BUA and Media Server

Asterisk acts as a Back-to-Back User Agent. It receives calls from Kamailio and handles:

- Dialplan (extensions.conf): from-kamailio context normalizes numbers to E.164 (+91XXXXXXXXXX)
- outbound context: strips + prefix, prepends Vocallabs routing prefix (45454), dials upstream
- from-vocallabs context: routes inbound calls back to Kamailio for DID-to-customer delivery
- CDR writing: cdr_mysql.conf writes call detail records to the cdr table in MySQL
- ARI (Asterisk REST Interface): on port 8088 — the Node.js API uses this for programmatic call control
- AMI (Asterisk Manager Interface): on port 5038 — used by the API for channel monitoring
- PJSIP endpoints: vocallabs-trunk (registration-based), kamailio (trusted local endpoint)

3.3 RTPEngine — Media Proxy

RTPEngine sits between Kamailio and all SIP endpoints. It terminates RTP (audio) streams, handles NAT traversal, and can bridge SRTP to RTP. Kamailio calls `rtpengine_offer()` and `rtpengine_manage()` to rewrite SDP connection addresses through RTPEngine's IP. Control socket: UDP 127.0.0.1:22222. RTP ports: 10000–20000.

3.4 MySQL Database Schema

Table	Purpose
subscriber	Customer SIP credentials (username, domain, password)
location	Active SIP registrations (contact URI, expiry)
customers	Customer accounts (email, balance, status)
trunks	SIP trunks per customer (credentials, max_channels)
did_inventory	DID numbers available for purchase
did_numbers	DIDs assigned to customers
cdr	Call Detail Records (duration, cost, billing status)
billing_transactions	Top-up and charge history
dispatcher	Asterisk node list for Kamailio load balancing
uacreg	Upstream registrations (Kamailio UAC module registers to providers)
dr_gateways	Dynamic routing gateway list (TATA, etc.)
address	IP whitelist for IP-authenticated customer trunks
acc / missed_calls	SIP accounting records

Section 4 — Outbound Call Flow (Step-by-Step)

This is what happens when a customer dials a number from their PBX or softphone:

Step	Actor	Action
1	Customer PBX	Sends SIP INVITE to Kamailio:5060 (destination: e.g. 9142436879)
2	Kamailio — AUTH	Challenges with 401 Unauthorized if not IP-authed; customer responds with credentials
3	Kamailio — AUTH	Validates digest against subscriber table; loads customer_id, trunk_id, max_channels
4	Kamailio — BALANCE	Reads sipaas:balance:{customer_id} from Redis; sends 402 if zero
5	Kamailio — CHANNELS	Reads sipaas:channels:{customer_id} from Redis; sends 503 if at limit
6	Kamailio — ROUTE	Calls rtpengine_offer() to anchor media. Adds X-Customer-ID + X-Trunk-ID headers. Increments channel counter
7	Kamailio → Asterisk	Forwards INVITE to Asterisk:5080 via dispatcher (group 1)
8	Asterisk — from-kamailio	Reads X-Customer-ID, X-Trunk-ID headers. Sets CDR fields
9	Asterisk — from-kamailio	Normalizes number: 10 digits → +91XXXXXXXXXX; 0XXXXXXXXXX → +91XXXXXXXXXX
10	Asterisk — outbound	Pattern matches +91. Sets CALLERID to 919228130351. Strips + from number
11	Asterisk — outbound	Prepends VOCALLABS_PREFIX (45454). DNIS becomes: 45454919142436879
12	Asterisk → Vocallabs	Sends INVITE sip:45454919142436879@vocallabs-trunk to 160.30.71.89:3300 with correct SDP (excludes audio)
13	Vocallabs	Routes 45454919142436879 to PSTN via their network. Sends 180 Ringing
14	Phone	Destination phone (9142436879) rings
15	Phone answers	Vocallabs sends 200 OK with SDP. Asterisk sends ACK. RTP audio flows
16	Hangup	BYE from either side → Kamailio decrements channel counter → Asterisk writes CDR → Billing engine

Key dialplan rule (extensions.conf):

```
exten => _+91.,1 → Set(DNIS=${VOCALLABS_PREFIX}${EXTEN:1})
```

For +919142436879: strips '+', prepends 45454 → sends 45454919142436879 to Vocallabs

Section 5 — Inbound Call Flow (Step-by-Step)

Step	Actor	Action
1	PSTN caller	Dials a DID number (e.g. +919228130352) that your customer owns
2	Vocallabs	Receives the PSTN call; sends SIP INVITE to Asterisk:5080 (from-vocallabs context)
3	Asterisk — from-vocallabs	Sets CDR fields. Dials PJSIP/{EXTEN}@kamailio to forward to Kamailio
4	Kamailio — INBOUND	Receives call from Asterisk (trusted, 127.0.0.1 passthrough)
5	Kamailio — DID lookup	Queries did_numbers WHERE number='{DID}' to find customer_id and trunk_id
6	Kamailio	Adds X-Customer-ID, X-Trunk-ID, X-DID headers. Calls rtpengine_offer()

Step	Actor	Action
7	Kamailio → Customer	Looks up customer registration in location table; forwards INVITE to customer PBX
8	Customer PBX	Receives call, rings the extension. Customer answers.
9	Audio flows	RTP bridged through RTPEngine between Vocallabs and Customer PBX

Section 6 — Customer Journey

Step	Action	What happens in the platform
1	Sign up on portal (http://YOUR_IP:8080)	Row created in customers table; JWT token issued
2	Top up balance	billing_transactions row added; customers.balance updated; Redis key set
3	Create a SIP Trunk	Row in trunks table; row in subscriber table (username=c{id}_ts, domain=YOUR_IP)
4	Get credentials	Portal shows: SIP Server, Port 5060, Username, Password
5	Buy a DID number	did_numbers row linked to trunk; balance deducted ■500; monthly renewal scheduled
6	Configure PBX/softphone	Enter: Server=YOUR_IP, Port=5060, User=c1_xxx, Pass=xxx
7	Softphone registers	Kamailio stores contact in location table; REGISTER 200 OK
8	Dial a number	Full outbound flow (Section 4) — call reaches PSTN
9	Receive a call on DID	Full inbound flow (Section 5) — call rings softphone
10	View CDRs on portal	REST API queries cdr table; portal displays call history + cost

Section 7 — Upstream Trunk Types

7.1 TATA (IP-Based Trunk)

- No SIP REGISTER required — TATA whitelists your PUBLIC_IP
- Kamailio drouting module routes calls directly to TATA_IP:TATA_PORT
- Configured in dr_gateways table and dr_rules table
- Asterisk pjsip.conf has a tata-trunk endpoint with no auth section
- Caller ID: set via TATA_CALLERID_PLACEHOLDER in pjsip.conf

7.2 Vocallabs / Provider2 (Registration-Based Trunk)

- Asterisk sends SIP REGISTER to Vocallabs server (160.30.71.89:3300) every 120 seconds
- Credentials stored in pjsip.conf: vocallabs-registration section
- Kamailio UAC module can also register to providers via the uacreg table
- Outbound calls use outbound_proxy=sip:160.30.71.89:3300 so Vocallabs receives them
- Vocallabs routing prefix 45454 must be prepended to every DNIS (set in .env as VOCALLABS_PREFIX)
- Caller ID set in pjsip.conf from_user=919228130351 (must be an approved CLI on the Vocallabs account)

Section 8 — Debugging Session: What We Fixed

During the session, the following issues were identified and resolved:

Fix 1 — Kamailio Crash: uacreg Table Version Mismatch

Symptom:

Kamailio container kept restarting. Error in logs:

```
ERROR: uac [uac_reg.c:1330]: invalid version 2 for table uacreg found, expected 5
```

Root cause:

The uacreg table was created with an older schema (version 2) but Kamailio 5.7 requires version 5, which adds two columns: contact_addr and socket.

Fix applied:

```
ALTER TABLE uacreg
  MODIFY auth_ha1 VARCHAR(128) NOT NULL DEFAULT '',
  MODIFY auth_proxy VARCHAR(255) NOT NULL DEFAULT '',
  ADD COLUMN contact_addr VARCHAR(255) NOT NULL DEFAULT '' AFTER reg_delay,
  ADD COLUMN socket VARCHAR(128) NOT NULL DEFAULT '' AFTER contact_addr;
UPDATE version SET table_version=5 WHERE table_name='uacreg';
```

Result:

Kamailio started and stayed up. All other services healthy.

Fix 2 — Wrong PUBLIC_IP in .env

The .env file had PUBLIC_IP=160.30.71.89, which is actually the Vocallabs SIP server IP, not our server's public IP. The server's actual public IP (as seen by Vocallabs) is 14.139.238.134.

```
# Before (wrong):  
PUBLIC_IP=160.30.71.89  
  
# After (correct):  
PUBLIC_IP=14.139.238.134
```

Fix 3 — Asterisk SDP Advertising Internal Docker IP

Symptom:

Calls connected (200 OK from Vocallabs) but were immediately hung up (BYE in under 1 second). The SIP INVITE's SDP was advertising the Docker internal IP:

```
c=IN IP4 192.168.65.3 ← unreachable from internet
```

Root cause:

Asterisk was running with network_mode: host inside Docker Desktop on Windows. The container bound to the Docker VM's internal IP (192.168.65.3). Vocallabs tried to send RTP to 192.168.65.3 — unreachable — so it hung up.

Fix applied — pjsip.conf transport section:

```
[transport-udp]  
type=transport  
protocol=udp  
bind=0.0.0.0:5080  
external_media_address=14.139.238.134  
external_signaling_address=14.139.238.134  
local_net=192.168.0.0/16  
local_net=172.16.0.0/12  
local_net=10.0.0.0/8
```

Result:

SDP now correctly shows: c=IN IP4 14.139.238.134

Fix 4 — Wrong Test Method (Bypassing Dialplan)

What we were doing (wrong):

```
# Bypasses Kamailio, auth, balance, and dialplan:  
channel originate PJSIP/919142436879@vocallabs-trunk ...
```

Correct test — through the full dialplan (outbound context):

```
# Goes through outbound context → adds 45454 prefix:  
channel originate Local/+919142436879@outbound application Wait 30
```

This correctly triggers the outbound dialplan which adds the 45454 Vocallabs routing prefix, sets the caller ID, and dials PJSIP/45454919142436879@vocallabs-trunk.

Current Status After All Fixes

Component	Status	Notes
Kamailio	Running stable	uacreg v5 migration applied
Asterisk	Running stable	Registered to Vocallabs
RTPEngine	Running	Listening on 127.0.0.1:22222
REST API	Running	Port 3000
Portal	Running	Port 8080
MySQL	Healthy	Port 3307 (host)
Redis	Healthy	Port 6379 (internal)
SDP IP	Fixed	Now advertises 14.139.238.134
Dialplan	Correct	outbound context adds 45454 prefix
Vocallabs PSTN	Pending	Needs verification at app.voicelink.co.in — account may need outbound PSTN enabled

Section 9 — Port Reference

Port	Protocol	Service	Who connects	Notes
5060	UDP + TCP	Kamailio SIP	Customer PBX / Softphones	Main SIP edge — must be internet-accessible
5080	UDP + TCP	Asterisk SIP	Kamailio only (internal)	Do NOT expose to customers
8088	TCP	Asterisk ARI	REST API (internal)	REST control interface
5038	TCP	Asterisk AMI	REST API (internal)	Manager interface
3000	TCP	REST API	Portal, external integrations	Node.js Express API
8080	TCP	Customer Portal	Browser	Next.js web dashboard
3001	TCP	Grafana	Admin browser	Monitoring dashboards
9090	TCP	Prometheus	Grafana, admin	Metrics endpoint
10000–20000	UDP	RTP Media	Remote SIP endpoints	Audio streams — must be internet-accessible
3300	UDP + TCP	Vocallabs	Asterisk (outbound)	Remote — Vocallabs SIP server, not local
3307	TCP	MySQL	API, Billing, Kamailio	Localhost only (127.0.0.1)
6379	TCP	Redis	Kamailio, API, Billing	Localhost only (127.0.0.1)
22222	UDP	RTPEngine ctrl	Kamailio (internal)	ng control protocol

Ports that MUST be reachable from the internet:

- * 5060 UDP/TCP -- for customer SIP phones to register and call
- * 10000-20000 UDP -- for RTP audio to flow (both inbound and outbound)
- * 5080 UDP -- for Vocallabs to send inbound calls (if you have inbound DIDs)

Section 10 — Local Setup: Step-by-Step Guide

Prerequisites

Requirement	Details
Docker Desktop	Windows/Mac: download from docker.com . Enable WSL2 backend on Windows.
Git	git-scm.com — to clone the repository
SIP Softphone	Zoiper (zoiper.com , free) or Linphone (linphone.org , free) for testing
VoIP Carrier Account	Vocallabs / Voicelink account at app.voicelink.co.in with outbound PSTN enabled
Public IP / Static IP	Your machine needs a reachable public IP, or use a VPS (recommended for production)
Router access	To set up port forwarding: 5060 UDP, 5080 UDP, 10000-20000 UDP

Recommendation for production:

Run on a Linux VPS (Ubuntu 22.04+) with a static public IP. Docker Desktop on Windows adds NAT complexity that requires careful `external_media_address` configuration. On a VPS, set `PUBLIC_IP` to the VPS's public IP and skip port forwarding — the IP is directly accessible.

Step 1 — Clone the Repository

```
git clone <your-repo-url> sipaas
cd sipaas
```

Step 2 — Configure .env

Copy the example and fill in all values:

```
cp .env.example .env
notepad .env # or use any text editor
```

Values to fill in:

Variable	What to put	Example
PUBLIC_IP	Your server's public IP (NOT Vocallabs IP)	14.139.238.134
DOMAIN	Same as PUBLIC_IP (or your domain)	14.139.238.134
VOCALLABS_IP	Vocallabs SIP server IP (from Vocallabs portal)	160.30.71.89
VOCALLABS_PORT	Vocallabs SIP port	3300
VOCALLABS_USER	Your Vocallabs account number	1769066634611
VOCALLABS_PASS	Your Vocallabs password	yourpassword
VOCALLABS_PREFIX	Routing prefix given by Vocallabs	45454
VOCALLABS_CALLER_ID	Your approved outbound CLI	919228130351
DB_USER / DB_PASS	MySQL credentials (choose any)	sipaas / StrongPass123!
DB_ROOT_PASS	MySQL root password	RootPass456!
REDIS_PASS	Redis password	RedisPass789!
JWT_SECRET	Random secret for API auth (32+ chars)	change-this-secret-key-now
AMI_USER / AMI_PASS	Asterisk Manager credentials	amiuser / AmiPassword!
ARI_USER / ARI_PASS	Asterisk REST API credentials	ariuser / AriPassword!

Step 3 — Find Your Public IP

Visit <https://whatismyip.com> or run in terminal:

```
# Windows PowerShell:
(Invoke-WebRequest ifconfig.me).Content
```

Set this IP as `PUBLIC_IP` in `.env`. Also set it in `asterisk/conf/pjsip.conf` as `external_media_address` and `external_signaling_address`.

Step 4 — Router Port Forwarding

In your router admin panel, add these port forwarding rules:

External Port	Internal Port	Protocol	Forward to
5060	5060	UDP + TCP	Your PC's LAN IP
5080	5080	UDP	Your PC's LAN IP
10000–20000	10000–20000	UDP	Your PC's LAN IP
3000	3000	TCP	Your PC's LAN IP (optional — for API access)
8080	8080	TCP	Your PC's LAN IP (optional — for portal access)

Step 5 — Start Database and Redis First

```
docker compose up -d mysql redis
# Wait 30 seconds for MySQL to initialize
Start-Sleep 30 # PowerShell
```

Step 6 — Apply the uacreg Database Migration

This is required for Kamailio 5.7 compatibility (schema version 2 → 5):

```
docker compose exec mysql mysql -usipaas -pYOUR_DB_PASS kamailio

# Then run this SQL:
ALTER TABLE uacreg
  MODIFY auth_ha1 VARCHAR(128) NOT NULL DEFAULT '',
  MODIFY auth_proxy VARCHAR(255) NOT NULL DEFAULT '',
  ADD COLUMN contact_addr VARCHAR(255) NOT NULL DEFAULT '' AFTER reg_delay,
  ADD COLUMN socket VARCHAR(128) NOT NULL DEFAULT '' AFTER contact_addr;
UPDATE version SET table_version=5 WHERE table_name='uacreg';
```

Step 7 — Start All Services

```
docker compose --profile sip up -d --build
# The --profile sip flag starts Kamailio, Asterisk, RTPEngine
# Other services (API, portal, billing, monitoring) start without the profile flag
```

Step 8 — Verify All Containers Are Running

```
docker compose ps

# Expected output:
NAME                STATUS
asterisk             Up
kamailio             Up
rtengine            Up
sipaas-api          Up      0.0.0.0:3000->3000/tcp
sipaas-billing       Up
sipaas-db            Up (healthy)
sipaas-grafana       Up      0.0.0.0:3001->3000/tcp
sipaas-portal        Up      0.0.0.0:8080->8080/tcp
sipaas-redis         Up (healthy)
```

Step 9 — Verify Vocallabs Registration

```
docker compose exec asterisk asterisk -rx 'pjsip show registrations'
```

Expected:

```
vocallabs-registration/sip:160.30.71.89:3300    vocallabs-auth    Registered
```

Step 10 — Access the Customer Portal

Open in browser:

```
http://localhost:8080          # if running locally  
http://YOUR_PUBLIC_IP:8080    # from another device
```

Step 11 — Create a Customer Account and SIP Trunk

- Register on the portal (or use the API)
- Top up balance — minimum **1** needed to allow calls
- Go to Trunks → Create Trunk → note the Username, Password, Domain
- Optionally: go to DIDs → Buy a DID number (requires balance)

Section 11 — Softphone Configuration (Zoiper / Linphone)

Zoiper Settings

Field	Value
Account type	SIP
Username	c1_1234567890 (from portal Trunks page)
Password	your-trunk-password (from portal)
Domain / SIP Server	YOUR_PUBLIC_IP or your domain
Port	5060
Transport	UDP
Outbound proxy	Leave empty
STUN	Disable

After configuring, Zoiper should show **Registered**. Then dial any mobile number (e.g. 9142436879 or +919142436879). The call flows through: Zoiper → Kamailio → Asterisk → outbound dialplan → Vocallabs → PSTN.

Section 12 — Admin Commands Reference

Asterisk CLI

```
docker compose exec asterisk asterisk -r
```

```
# Inside Asterisk CLI:
```

```
pjsip show registrations      -- check Vocallabs registration
pjsip show endpoints         -- show all endpoints
pjsip show contacts          -- show active contacts
core show channels           -- show active calls
pjsip set logger on          -- enable SIP packet logging
core set verbose 5           -- enable verbose logging
dialplan show from-kamailio  -- show outbound dialplan
cdr show status              -- CDR module status
```

Kamailio Admin

```
docker compose exec kamailio kamctl ul show      -- active registrations
docker compose exec kamailio kamcmd dlg.list     -- active calls
docker compose exec kamailio kamcmd dispatcher.reload -- reload Asterisk nodes
docker compose exec kamailio kamcmd uac.reg_reload -- reload UAC registrations
docker compose exec kamailio kamcmd uac.reg_info  -- check upstream reg status
```

Live Monitoring

```
docker compose logs -f asterisk          -- live Asterisk log (SIP trace)
docker compose logs -f kamailio          -- live Kamailio log
docker compose logs -f sipaas-api        -- live API log
docker compose logs -f sipaas-billing    -- live billing log
```

Test a Call (Admin)

```
# Test through the full outbound dialplan (correct method):
docker compose exec asterisk asterisk -rx \
    'channel originate Local/+919142436879@outbound application Wait 30'

# Then watch the SIP trace:
docker compose logs asterisk --since=10s --tail=200
```

Section 13 — Firewall Rules (Linux VPS)

```
# SIP signaling
ufw allow 5060/udp
ufw allow 5060/tcp
ufw allow 5080/udp

# RTP audio (must match RTP_MIN_PORT / RTP_MAX_PORT in .env)
ufw allow 10000:20000/udp

# Platform access
ufw allow 3000/tcp    # REST API
ufw allow 8080/tcp    # Portal

# Monitoring (restrict to your IP in production)
ufw allow 3001/tcp    # Grafana
ufw allow 9090/tcp    # Prometheus

ufw enable
```

Section 14 — Troubleshooting

Symptom	Likely Cause	Fix
Kamailio keeps restarting	uacreg table version mismatch	Run the ALTER TABLE migration (Section 10, Step 6)
Softphone can't register (401)	Wrong domain in subscriber table	Ensure subscriber.domain matches PUBLIC_IP in .env
402 Insufficient Balance	Customer balance is 0 in Redis	Top up via portal or: redis-cli SET sipaas:balance:1 100
503 Channel Limit	Customer at max concurrent calls	Increase max_channels in trunks table or hang up existing calls
Call connects but no audio	RTP ports blocked or wrong external IP	Check external_media_address in pjsip.conf; open ports 10000-20000 U
Vocallabs BYE in <1 second	Account not provisioned for outbound	On the SIPp.voicelink.co.in — enable outbound, check balance, verify CL
603 Declined from Vocallabs	CLI not authorized or wrong number	Check CLI 919228130351 is approved; try different number format
Routing cannot resolve IP	TATA placeholder IP in dr_gateways	Update dr_gateways table with real TATA IP, or ignore if using Vocallab

Symptom	Likely Cause	Fix
Kamailio dispatcher 401 on OPTIONS	Kamailio endpoint misconfigured in sip.conf	Ensure [kamailio] endpoint has correct aors and trust_id settings
docker compose ps shows Restarting	Container crash — check logs	docker compose logs <service> --tail=50
Portal shows blank / can't login	API not connected to DB	docker compose logs sipaas-api; check DB_HOST env var

Quick health check command:

docker compose ps | Check all containers show 'Up' or 'healthy'

docker compose exec asterisk asterisk -rx 'pjsip show registrations' | Should show 'Registered'

docker compose exec kamailio kamcmd dispatcher.list | Should show Asterisk node as Active